

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Estrutura de Dados II

Professor: MSc. Jonathan Henrique Jeremias Souza

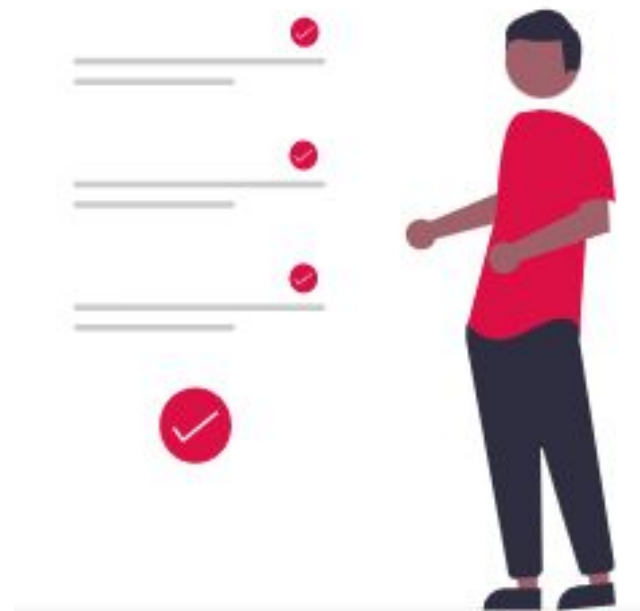
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Revisão do conceito de árvores
- Percorrer árvores
- Conceito de árvores balanceadas



PERCURSOS EM ÁRVORES

Introdução

- Os percursos em árvores binárias normalmente são classificados em pré-ordem, em ordem, pós-ordem e percurso em largura, cada um seguindo regras próprias de navegação.
- A diferença principal entre esses percursos está no momento em que o nó atual é processado em relação aos seus filhos esquerdo e direito.
- Compreender essa diferença é fundamental para interpretar corretamente o comportamento de algoritmos que utilizam árvores.

PERCURSOS EM ÁRVORES

Percurso em pré-ordem

- No percurso em pré-ordem, o algoritmo visita primeiro o nó atual antes de acessar recursivamente os filhos esquerdo e direito da árvore.
- Essa estratégia é útil quando desejamos processar o elemento principal antes de analisar suas subdivisões, sendo comum em cópias estruturais e serialização de árvores.
- A sequência lógica do algoritmo pode ser resumida como raiz, subárvore esquerda e subárvore direita.

PERCURSOS EM ÁRVORES

Percurso em pré-ordem

- Antes de observar o código do percurso em pré-ordem, é importante compreender que o algoritmo normalmente utiliza recursividade para navegar pela estrutura.
- A recursividade permite que a própria função seja chamada novamente para processar as subárvores descendentes de cada nó visitado.
- Esse comportamento simplifica bastante a implementação de algoritmos em árvores binárias.

PERCURSOS EM ÁRVORES

Percurso em pré-ordem

- O algoritmo abaixo implementa o percurso em pré-ordem utilizando recursividade para navegar entre os nós da árvore binária.
- Observe que o processamento do nó ocorre antes das chamadas recursivas para os filhos esquerdo e direito.
- Essa posição do processamento define exatamente o comportamento da pré-ordem.

```
public void preOrdem(No raiz) {  
    if (raiz != null) {  
        System.out.println(raiz.valor);  
        preOrdem(raiz.esquerda);  
        preOrdem(raiz.direita);  
    }  
}
```

PERCURSOS EM ÁRVORES

Percurso em pré-ordem

- A primeira verificação realizada pelo algoritmo garante que o nó atual realmente exista antes de tentar acessar seus dados ou descendentes.
- O valor armazenado no nó é exibido imediatamente, demonstrando que o processamento ocorre antes da exploração das subárvores.
- Depois disso, o algoritmo visita recursivamente o filho esquerdo e posteriormente o filho direito.

PERCURSOS EM ÁRVORES

Percurso em ordem

- No percurso em ordem, o algoritmo visita primeiro a subárvore esquerda, depois o nó atual e finalmente a subárvore direita.
- Esse percurso possui enorme importância em árvores binárias de busca porque produz os elementos em sequência ordenada.
- Por essa razão, o percurso em ordem é amplamente utilizado em sistemas que precisam listar informações organizadas.

PERCURSOS EM ÁRVORES

Percurso em ordem

- Ao implementar o percurso em ordem, a principal diferença está no momento em que o nó atual será processado durante a navegação.
- Nesse caso, o processamento ocorre entre as visitas às subárvores esquerda e direita.
- Essa mudança aparentemente simples altera completamente o resultado produzido pelo algoritmo.

PERCURSOS EM ÁRVORES

Percurso em ordem

- O código abaixo mostra a implementação clássica do percurso em ordem utilizando chamadas recursivas para navegar pela árvore.
- Observe atentamente a posição da impressão do valor em relação às chamadas recursivas.
- Essa organização define o comportamento específico desse tipo de percurso.

```
public void emOrdem(No raiz) {  
    if (raiz != null) {  
        emOrdem(raiz.esquerda);  
        System.out.println(raiz.valor);  
        emOrdem(raiz.direita);  
    }  
}
```

PERCURSOS EM ÁRVORES

Percurso em ordem

- Como o algoritmo visita primeiro todos os elementos menores localizados à esquerda, os valores acabam sendo exibidos em ordem crescente em árvores binárias de busca.
- Esse comportamento transforma o percurso em ordem em uma estratégia extremamente útil para ordenação e exibição organizada de dados.
- Muitos sistemas utilizam esse tipo de percurso para gerar relatórios e listagens automaticamente organizadas.

PERCURSOS EM ÁRVORES

Percurso em pós-ordem

- No percurso em pós-ordem, o algoritmo visita primeiro as subárvores esquerda e direita antes de processar o nó atual.
- Essa abordagem é útil em situações nas quais os descendentes precisam ser processados antes do elemento pai.
- Operações como remoção de estruturas e liberação de memória frequentemente utilizam esse tipo de percurso.

PERCURSOS EM ÁRVORES

Percurso em pós-ordem

- O algoritmo abaixo demonstra a implementação recursiva clássica do percurso em pós-ordem em árvores binárias.
- Observe que a impressão do valor ocorre somente após o processamento completo das subárvores.
- Essa característica define o comportamento do algoritmo.

```
public void posOrdem(No raiz) {  
    if (raiz != null) {  
        posOrdem(raiz.esquerda);  
        posOrdem(raiz.direita);  
        System.out.println(raiz.valor);  
    }  
}
```

PERCURSOS EM ÁRVORES

Percurso em pós-ordem

- O processamento final do nó garante que todos os descendentes sejam tratados antes do elemento pai da estrutura.
- Esse comportamento é particularmente importante em algoritmos que precisam eliminar estruturas de baixo para cima.
- Dessa forma, evitam-se acessos inválidos a nós já removidos ou desalocados.

PERCURSOS EM ÁRVORES

Percurso em largura

- Diferentemente dos percursos recursivos tradicionais, o percurso em largura visita os nós nível por nível dentro da árvore.
- Primeiramente são visitados todos os elementos do nível superior e somente depois os elementos dos níveis inferiores.
- Esse comportamento normalmente exige o uso de uma fila auxiliar para controlar a sequência de visita.

PERCURSOS EM ÁRVORES

Percurso em largura

- Como o percurso em largura trabalha com controle sequencial dos níveis da árvore, o algoritmo precisa armazenar temporariamente os nós pendentes de visita.
- A estrutura mais adequada para isso é a fila, pois ela respeita a ordem de chegada dos elementos.
- Esse mecanismo garante que os nós sejam visitados corretamente nível por nível.

PERCURSOS EM ÁRVORES

Percurso em largura

- O algoritmo abaixo utiliza uma fila para visitar os nós da árvore em sequência horizontal, começando pela raiz.
- Os filhos de cada nó visitado são inseridos na fila para processamento posterior.
- Esse comportamento produz o percurso por níveis.

```
Queue<No> fila = new LinkedList<>();
fila.add(raiz);

while (!fila.isEmpty()) {
    No atual = fila.remove();
    System.out.println(atual.valor);

    if (atual.esquerda != null) {
        fila.add(atual.esquerda);
    }

    if (atual.direita != null) {
        fila.add(atual.direita);
    }
}
```

PERCURSOS EM ÁRVORES

Percurso em largura

- A fila controla quais nós ainda precisam ser processados, garantindo que os elementos mais antigos sejam visitados primeiro.
- Sempre que um nó é removido da fila, seus filhos são adicionados ao final da estrutura para processamento posterior.
- Esse comportamento mantém corretamente a navegação em níveis horizontais.

PERCURSOS EM ÁRVORES

Árvores Balanceadas

- Embora árvores binárias possam oferecer desempenho eficiente, isso depende diretamente da forma como seus elementos estão distribuídos estruturalmente.
- Quando muitos nós ficam concentrados em apenas um dos lados da árvore, ocorre o desbalanceamento estrutural.
- Esse problema pode degradar significativamente o desempenho das operações de busca, inserção e remoção.

PERCURSOS EM ÁRVORES

Árvores Balanceadas

- Em uma árvore balanceada, as operações normalmente percorrem poucos níveis até encontrar um elemento desejado.
- Por outro lado, árvores extremamente desbalanceadas podem se comportar praticamente como listas encadeadas, aumentando drasticamente o número de acessos necessários.
- Esse aumento de profundidade reduz a eficiência algorítmica da estrutura.

PERCURSOS EM ÁRVORES

Árvores Balanceadas

- As árvores balanceadas foram desenvolvidas para manter a estrutura relativamente equilibrada mesmo após diversas inserções e remoções.
- O objetivo principal dessas estruturas é impedir que a árvore cresça excessivamente para apenas um lado.
- Com isso, o desempenho das operações permanece eficiente mesmo em grandes volumes de dados.

PERCURSOS EM ÁRVORES

Árvores Balanceadas

- A árvore AVL é um tipo de árvore binária balanceada criada para garantir que a diferença de altura entre subárvores permaneça controlada.
- Sempre que o balanceamento é comprometido, a estrutura realiza rotações automáticas para restaurar o equilíbrio.
- Esse comportamento mantém as operações eficientes mesmo após inúmeras modificações estruturais.

PERCURSOS EM ÁRVORES

Árvores Balanceadas

- O fator de balanceamento representa a diferença entre a altura da subárvore esquerda e da subárvore direita de um nó.
- Nas árvores AVL, normalmente aceita-se apenas fatores de balanceamento iguais a -1, 0 ou 1.
- Valores fora desse intervalo indicam necessidade de reorganização estrutural através de rotações.

PERCURSOS EM ÁRVORES

Rotações em árvores AVL

- Quando uma árvore AVL perde o equilíbrio, algumas regiões da estrutura tornam-se excessivamente profundas em relação às demais.
- As rotações reorganizam os nós da árvore sem violar as propriedades da árvore binária de busca.
- Esse processo reduz novamente a profundidade estrutural e restaura o desempenho eficiente das operações.

PERCURSOS EM ÁRVORES

Rotações em árvores AVL

- A rotação simples à direita é utilizada quando ocorre excesso de profundidade na subárvore esquerda do nó problemático.
- Durante a rotação, um dos descendentes assume posição superior na hierarquia da árvore.
- Esse ajuste reduz a profundidade excessiva sem perder a organização dos elementos.

PERCURSOS EM ÁRVORES

Rotações em árvores AVL

- A rotação simples à esquerda ocorre em situações opostas, quando a subárvore direita cresce excessivamente em relação à esquerda.
- O processo reorganiza os nós deslocando o descendente direito para uma posição mais alta na estrutura.
- Após a rotação, a árvore volta a apresentar distribuição mais equilibrada.

PERCURSOS EM ÁRVORES

Rotações em árvores AVL

- Em algumas situações, uma única rotação não é suficiente para restaurar corretamente o balanceamento estrutural da árvore.
- Nesses casos, utilizam-se rotações duplas, combinando rotações simples em sequência para corrigir o problema.
- As rotações duplas aparecem principalmente quando o desbalanceamento ocorre em direções cruzadas.

PERCURSOS EM ÁRVORES

Atividade

- Desenhe manualmente uma árvore binária contendo pelo menos sete elementos organizados em diferentes níveis hierárquicos dentro da estrutura.
- Depois disso, execute manualmente os percursos em pré-ordem, em ordem e pós-ordem registrando a sequência produzida em cada caso.
- Ao final da atividade, compare os resultados obtidos para compreender como pequenas mudanças no algoritmo alteram completamente a ordem de visita.

PERCURSOS EM ÁRVORES

Atividade

- Monte uma árvore propositalmente desbalanceada inserindo elementos em ordem crescente e observe como a profundidade estrutural aumenta progressivamente.
- Depois disso, tente reorganizar manualmente os nós utilizando rotações simples para aproximar a estrutura de uma árvore balanceada.
- Esse exercício ajuda a compreender visualmente por que árvores AVL conseguem manter desempenho eficiente.

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- PUGA, Sandra; RISSETTI, Gerson. Lógica de programação e estruturas de dados com aplicações em Java. 2. ed. São Paulo: Pearson, 2009.
- CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Elsevier.
- TENENBAUM, Aaron M. Estruturas de dados usando C. São Paulo: Makron Books.
- GOODRICH, Michael T.; TAMASSIA, Roberto. Estruturas de dados e algoritmos em Java. Porto Alegre: Bookman.
- Documentação Java Oracle. Disponível em: <https://docs.oracle.com/javase/>